P.Vishwa Nandan(192473015)

TASK B4 – FINAL ENGINEERING DECISION (5 MARKS)

1. Aim

To design a practical database migration strategy for the university transport office when the number of bus-pass records increases across 12 campuses.

The task implements file-to-database migration using JDBC batch processing and transactions. It also uses DatabaseMetaData to inspect database information and evaluates the most suitable storage solution for production.

2. Problem Description

The transport office currently stores bus-pass records in files. The system is now expected to serve:

- 12 university campuses.
- 100 times the current number of passes.
- Multiple counters issuing passes simultaneously.
- Large numbers of records requiring searching and updating.

A simple file-based system may become difficult to maintain at this scale.

Therefore, the existing pass records must be migrated into a shared MySQL database using batch processing and transactions.

3. B4.1 – Implement a Practical Strategy (2 Marks)

3.1 Aim

To read the existing bus-pass file, validate each record, and insert valid records into MySQL using PreparedStatement batches and a single transaction.

Invalid records must be reported separately without stopping the migration of other valid records.

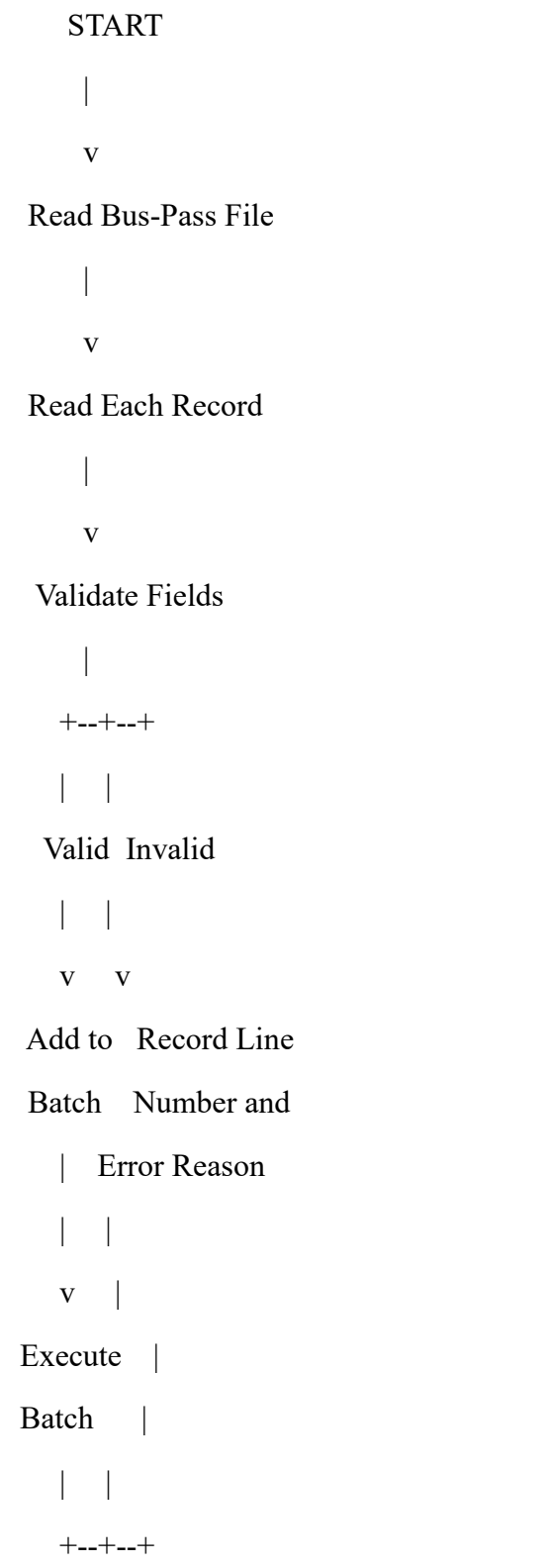
3.2 Migration Process

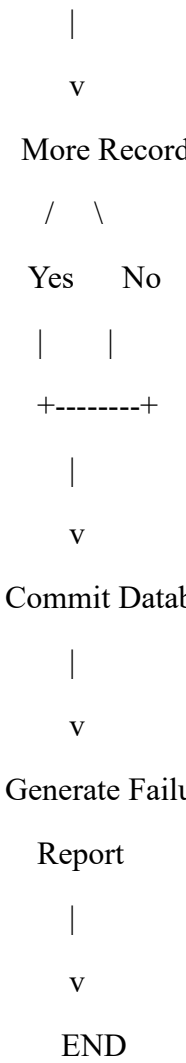
The migration follows these steps:

1. Read the existing CSV file line by line.
2. Check whether each record contains exactly five fields.
3. Validate pass ID, registration number, name, route number, and amount.
4. Store valid records for database insertion.
5. Store invalid records with their line number and failure reason.

6. Insert valid records into MySQL in batches of 1,000.
7. Commit all valid records in one transaction.
8. Generate a failure report.

Migration Flow





3.3 MySQL Database Setup

Use the database and table created in Task B3.

SQL Program – migration_database.sql

```
CREATE DATABASE IF NOT EXISTS assignment_persistence;

USE assignment_persistence;

CREATE TABLE IF NOT EXISTS bus_pass (

    pass_id VARCHAR(20) PRIMARY KEY,

    reg_no VARCHAR(20) NOT NULL,
```

```
    name VARCHAR(100) NOT NULL,

    route_no INT NOT NULL,

    amount DECIMAL(10,2) NOT NULL

) ENGINE = InnoDB;
```

3.4 Sample Existing Bus-Pass File

Create a file named:

existing-passes.csv

Add the following records:

P1001,192311001,Arun,12,4500.00

P1002,192311002,Divya,7,3800.00

P1003,192311003,Ravi,5,4200.00

P1004,192311004,Meena,INVALID,4000.00

P1005,192311005,Kumar,8,ABC

P1006,192311006,Anu,9,3500.00

P1007,192311007,John,4,3900.00

P1008,192311008, ,6,3200.00

This file contains both valid and invalid records.

Expected validation:

- P1001 – Valid.
- P1002 – Valid.
- P1003 – Valid.
- P1004 – Invalid route number.
- P1005 – Invalid amount.
- P1006 – Valid.
- P1007 – Valid.

- P1008 – Empty student name.

3.5 Java Program – PassMigration.java

Explanation

Step 1 – Read the Existing File

BufferedReader reads the CSV file line by line.

Step 2 – Validate Records

The validate() method checks:

- Number of fields.
- Pass ID.
- Registration number.
- Student name.
- Numeric route number.
- Numeric and non-negative amount.

Step 3 – Separate Valid and Invalid Records

Valid records are added to validRecords.

Invalid records are stored in failures with their line numbers and reasons.

Step 4 – Insert Using JDBC Batch

Valid records are inserted using PreparedStatement.

The program executes a batch after every 1,000 records.

Step 5 – Use a Transaction

All valid records are inserted inside one transaction.

If a database insertion fails, rollback() cancels the database changes made by the migration.

Step 6 – Generate Failure Report

Invalid file records are written into migration-failures.txt.

This allows the transport office to correct and review rejected records.

Important implementation note: This sample uses a simple comma-separated format and assumes names do not contain commas. For production CSV files that allow quoted commas, use a proper CSV parser. For very large migration files, valid records and failure reports should also be processed incrementally rather than keeping the entire dataset in memory.

3.6 Expected Output

Migration completed.

Total lines processed: 8

Valid records inserted: 5

Invalid records: 3

Failure report: migration-failures.txt

3.7 Expected Failure Report

File: migration-failures.txt

BUS-PASS MIGRATION FAILURE REPORT

Line 4: Invalid route number | P1004,192311004,Meena,INVALID,4000.00

Line 5: Non-numeric amount | P1005,192311005,Kumar,8,ABC

Line 8: Empty student name | P1008,192311008, ,6,3200.00

3.8 Verify the Migrated Records

Run the following SQL commands in MySQL Workbench:

```
USE assignment_persistence;

SELECT * FROM bus_pass;

SELECT COUNT(*) FROM bus_pass;
```

Expected record count:

5

The five valid records should be present in the database.

4. B4.2 – Inspect the Database (1 Mark)

4.1 Aim

To use JDBC DatabaseMetaData to identify the database product, JDBC driver version, and available tables.

4.2 Java Program – DatabaseInspector.java

4.3 Explanation

The program uses:

DatabaseMetaData md = con.getMetaData();

DatabaseMetaData provides information about the connected database.

The following methods are used:

- getDatabaseProductName() – Returns the database product name.
- getDatabaseProductVersion() – Returns the database version.
- getDriverName() – Returns the JDBC driver name.
- getDriverVersion() – Returns the JDBC driver version.
- getTables() – Retrieves the tables available in the database.

4.4 Expected Output

The exact versions depend on the installed MySQL server and JDBC driver.

Example:

Database Product: MySQL

Database Version: 8.x

JDBC Driver: MySQL Connector/J

Driver Version: 9.x

Available Tables:

bus_pass

4.5 Why Is DatabaseMetaData Useful?

DatabaseMetaData is useful when moving the application to another server because:

1. It identifies which database product is connected.
2. It helps identify differences between database versions.
3. It identifies the JDBC driver being used.
4. It allows the application to inspect available tables.
5. It helps developers check database compatibility before deployment.

For example, if the application is moved from one MySQL server to another, developers can inspect the new database version and driver before running the migration or application.

5. B4.3 – Production Decision (2 Marks)

5.1 Aim

To select the most suitable storage approach for the transport office when it expands to 12 campuses with 100 times the current number of bus passes and multiple counters operating simultaneously.

5.2 Comparison of Storage Approaches

1. Text Files

Text files store records as readable lines separated by delimiters.

Advantages:

- Simple to create and read.
- Easy to transfer between systems.
- Does not require a database server.
- Suitable for small datasets and backups.

Disadvantages:

- Searching large files requires scanning records.
- Multiple counters may create file access conflicts.
- Concurrent updates are difficult to manage safely.
- Data consistency and security are harder to enforce.

2. Java Serialization

Java serialization stores Java objects in binary form.

Advantages:

- Saves and restores Java objects.
- Can preserve object relationships.
- Useful for certain Java-specific applications.

Disadvantages:

- Not easily readable by other programming languages.
- Managing simultaneous updates is difficult.
- Version compatibility can become a problem.
- Not suitable as the main shared storage system for multiple campuses.

3. Relational Database – MySQL

A relational database stores records in structured tables.

Advantages:

- Supports multiple users and simultaneous operations.
- Provides transactions using commit and rollback.
- Supports fast searching through indexes.
- Provides primary keys and constraints.
- Supports centralized storage for all campuses.
- Allows access control and backup management.

Disadvantages:

- Requires database administration.
- Needs server configuration and maintenance.
- Database availability and backups must be managed.

5.3 Final Production Decision

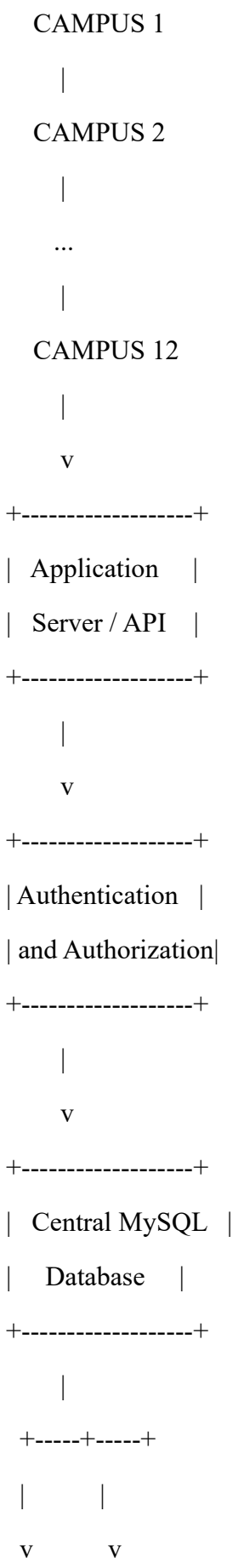
For the expanded transport office, I would select:

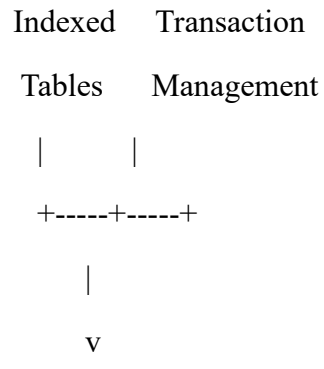
A Centralized MySQL Database with Indexed Tables, Transactions, and Role-Based Access Control.

Reasons:

1. **Concurrent Counters:** Multiple counters can issue passes simultaneously through database connections and transaction management.
2. **Fast Searching:** Indexes on pass ID, registration number, and route number can improve searching performance.
3. **Data Safety:** Transactions prevent incomplete operations from being treated as successful.
4. **Centralized Management:** All 12 campuses can access a shared database instead of maintaining separate files.
5. **Data Integrity:** Primary keys and constraints prevent duplicate pass IDs and invalid database values.
6. **Security:** User roles and permissions can restrict database operations to authorized employees.
7. **Scalability:** The database can be expanded with additional server resources and suitable database architecture as the number of records grows.

5.4 Proposed Production Architecture


CAMPUS 1
CAMPUS 2
...
CAMPUS 12
Application Server / API
Authentication and Authorization
Central MySQL Database



5.5 Alternative – Separate Serialized Files for Each Campus

Another possible solution is to maintain one serialized file for each campus.

For example:

- Campus 1 – campus1.dat
- Campus 2 – campus2.dat
- Campus 3 – campus3.dat
- Campus 12 – campus12.dat

Advantages

1. Each campus can manage its own records.
2. Local file access may be simple.
3. A campus can continue working independently when disconnected from the central network.

Disadvantages

1. Searching across all campuses requires accessing multiple files.
2. Duplicate pass IDs may occur between campuses unless coordinated.
3. Simultaneous file updates require additional locking.
4. Centralized reports become more difficult.
5. Maintaining consistent records across 12 files is challenging.

For a global search, the system may need to inspect all 12 files.

If each campus has n records, the total number of records is:

Total records = 12 × n

A sequential search across all campus files can require:

O(12n)

Since 12 is a constant, this is O(n) in asymptotic notation, but the system still has to access and inspect records distributed across all 12 files.

A centralized database with suitable indexes is a more practical solution for shared searching and concurrent updates.

6. Complexity Analysis

Let:

- N = Total number of bus-pass records.
- b = Batch size.
- k = Number of campuses.

File-Based Migration

Reading the file and validating each record requires:

Time Complexity: O(N)

The migration processes each record once.

Batch Database Insertion

For N records and a batch size of b:

Number of batches = ceil(N / b)

The total insertion work is O(N), with fewer database communication operations than one-record-at-a-time insertion.

Database Search

With a suitable index on the searched column:

Typical indexed lookup: O(log N)

The actual performance depends on the database index, query plan, and storage engine.

Separate Campus Files

Searching all campus files requires inspecting records across all campuses:

O(k × n)

where k is the number of campuses and n is the number of records per campus.

7. Overall Engineering Observation

The migration experiment demonstrates how existing file-based records can be transferred into MySQL using validation, PreparedStatement batches, and transactions.

Invalid records are reported separately, while valid records are inserted into the database.

DatabaseMetaData provides information about the connected database and JDBC driver, which helps when deploying the application on another server.

For the expanded 12-campus transport office, a centralized MySQL database is more suitable than separate text or serialized files because it supports concurrent access, transactions, indexed searching, and centralized management.

8. Final Result

A Java-based migration program was developed to read the existing bus-pass file, validate records, and insert valid records into MySQL using batches and a transaction.

Invalid records were recorded in a separate failure report with their line numbers and reasons.

A DatabaseMetaData program was implemented to retrieve the database product, database version, JDBC driver version, and available tables.

Finally, text files, Java serialization, and MySQL were compared. A centralized MySQL database with indexing, transactions, and role-based access control was selected as the most suitable production solution for the university's 12 campuses and multiple counters.